

Splitting app.py — a report on #418

19 September 2026. Nine pull requests in kuantorflow, nine in kuantorflow_automation. No route, endpoint, template, page or schema behaves differently at the end of it.

1. The headline

app.py was **4,917 lines and 219 KB** in one file — more than twice the next largest module in the repo, and the only file nobody could hold in their head. It is now **538 lines**.

	before	after
app.py	4,917 lines, 219 KB	538 lines, 21 KB
modules holding the web app	1	6
@app.route in one file	38	8
Python files in the repo root	19	13
offline tests	2,082	2,109
tests with a real database	2,223	2,250
template changes	—	none

The line total across the six modules is 5,259 against 4,917. The extra 342 lines are module docstrings and import headers: no application logic was written, deleted or rewritten during #418.

2. What the file was

Read off the pre-split file rather than guessed:

- **162** top-level functions and classes
- **66** top-level assignments
- **38** routes and **8** context processors / before_request handlers
- eleven distinct concerns interleaved, in no particular order

The blocks, in the order they appeared in the file:

block	~lines
Boot and configuration	260
Identity and permissions	390
Saving and duplicate marks	260
Mykola chat	800
Auth routes and context processors	220

Topic and game icons	95
Account deletion and settings	100
Card routes	775
Spending guards	230
The games chassis and ten rounds	1,470
Building a topic from an idea (#406)	280
The quiz	230

3. What it is now

Six modules, and the dependency runs **one way**:

```
web.py <- icons.py <- cards.py <- rounds.py <- chat.py <- app.py
```

module	lines	KB	defs	routes	proc.	holds
web.py	728	33	28	0	0	the Flask object, configuration, identity and permissions, the two spending guards, the SSE frame
icons.py	119	5	5	0	0	topic and activity icons, as Jinja filters
cards.py	1,364	65	27	16	0	the deck, every card read and write, <code>_save_and_log()</code> , #406's topic builder
rounds.py	1,550	73	41	5	1	the games chassis, ten rounds, the quiz
chat.py	960	41	44	9	2	Mykola: the agent import, the singleton, the chat routes, the chat logs
app.py	538	21	17	8	5	the keyword gate, Google sign-in, settings, account deletion, four side-effect imports

Nothing may import `app.py`. That is the property the whole split rests on: the moment a feature module imports it, the route table comes along behind it and the separation is undone. It is asserted, per module, in `tests/test_web_is_the_shared_module.py`.

`app.py` imports `cards`, `chat`, `icons` and `rounds` **for their side effects** — routes, context processors, Jinja filters and `GAME_ROUNDS` all register themselves — and asks them for nothing except `cards._save_and_log()` and `cards.DEFAULT_TOPIC`, which it reaches through the module.

The admission rule for `web.py`

`web.py` takes only what two or more feature modules need. Without a stated rule it becomes `app.py` under another name, and that is the one way this refactor could fail while every test stayed green.

It decided the two borderline cases:

- the **spending guards moved there**, because `_generation_refusal()` is asked by #237's reader *and* #406's topic builder — two different features;

- ``GENERATED_TEXT_KEY`` stayed with the round, because only the reader reads it.

`icons.py` exists for the same reason in reverse: the topic tiles belong to `cards.py` and the activity tiles to `rounds.py`, so the icons could live in neither — and giving them 119 lines of their own kept `web.py` from absorbing them.

4. The decision that governed everything: route modules, not blueprints

Blueprints are the idiomatic Flask answer and they **rename every endpoint**. `url_for('index')` becomes `url_for('cards.index')`. The templates use **32 distinct endpoints across roughly 90 call sites**, and a missed one is a 500 at render time — not a failed import, not a red test.

Issue #250 is a live reminder that endpoint identity is load-bearing here: `/quiz` and `/quiz/<topic>` are deliberately two endpoints, because `url_for` otherwise picks the path converter and the multi-topic URL becomes unbuildable.

So: **route modules registered against the one ``app`` object**, keeping every endpoint name. Less idiomatic, and it needs an answer for the import cycle — which is what `web.py` is.

The result of that decision: zero template edits across nine PRs.

5. The prerequisites — two PRs that moved no application code

5.1 Prove the suite stays offline (automation#153)

`conftest.py` held the promise that no test touches a real database, and it kept that promise by stubbing names **on the ``app`` module**. That worked only while every database call lived in `app.py`.

Simulating a single moved helper flagged **21 routes**, every one answering 200 while reading a real database, with the suite green. This project had already paid that bill once: tests incrementing the live `text_generation_usage` counter until an account hit its daily ceiling and unrelated tests began failing for reasons nobody could see.

`tests/test_suite_stays_offline.py` patches `utils.get_db_connection` to raise and walks every route derived from the app's own URL map. **The attempt is the assertion, not the status code** — several read paths catch a dead database and degrade to an empty page on purpose, so a route that reached a real connection can still answer 200. Asserting on the response was this test's first version, and it passed while the very move it exists to catch was simulated against it.

5.2 The patch-target migration (#436, two PRs)

`from utils import get_topics` **copies** the function into the importer's namespace. There are then two names for one function, and the suite stubbed the copy. Move a caller to a new module and the stub stops reaching it — silently.

Measured cost before starting: **343 ``monkeypatch.setattr(app_module, ...)`` calls across 118 test files, 54 distinct names.**

The change: `import utils` and `utils.get_topics(...)`, resolved at call time, so one `monkeypatch.setattr("utils.get_topics", fake)` reaches every caller in every module — including ones not yet written.

- **59** call sites rewritten in `app.py`
- **43** test files plus `conftest.py` repointed
- offline and database tiers unchanged; `--collect-only` identical to `main`

This is the step that made every later move cheap. It was filed as its own ticket rather than discovered inside a move, because a mistake in `conftest.py` breaks all 2,073 tests at once.

6. The steps

Each row is one code PR and one test PR. "Churn" is the number of test references the move required — the whole cost of the move, measured.

step	PR	moved	app.py after	churn
3	#435	the Flask object and configuration to <code>web.py</code>	4,813	0
4	#438	identity and permission helpers to <code>web.py</code>	4,529	28
5	#439	spending guards to <code>web.py</code>	4,361	6
6	#440	games chassis, ten rounds, quiz to <code>rounds.py</code>	2,860	28
7	#441	card routes to <code>cards.py</code> , icons to <code>icons.py</code>	1,435	62
8	#443	Mykola to <code>chat.py</code>	538	41
—	#444	six console scripts to <code>scripts/</code>	538	12

Step 3 — `web.py` (#435)

The Flask object, the secret, the session lifetime, the environment readers, `ADMIN_EMAILS`, the OAuth client. **No test changes at all**, because every *user* of those names stayed in `app.py`.

Step 4 — identity and permissions (#438)

Thirty names, 317 lines: who is asking, whether they are blocked, an admin or the author of a card, and what they are told when the answer is no.

This is the slice that had already been attempted and reverted at 278 failures, before #436 existed. With the patch targets settled it cost **28 patch sites and nothing else** — and the counterfactual was measured, not assumed: the move alone produces 39 failures, and repointing those 28 clears every one.

167 call sites in `app.py` were rewritten to `web.X` by a `tokenize` pass rather than a regex, so no string, comment or docstring was touched. One comment reading `{% if is_admin %}` — about a *template* variable — would have been silently corrupted otherwise.

Step 5 — spending guards (#439)

Seventeen names: #388's lookup ceilings, #237's generation ceilings, their refusal wording, `_generation_available()`.

Not originally planned as its own step. The dependency pass found that the games block needed **exactly one thing** from `app.py` — `_generation_refusal()` — and that #406's topic builder needed it too. One guard, two features, so it could live in neither caller. Moving it first made `rounds.py` a clean one-directional move.

Cost: **3 failures, 6 edits.**

Step 6 — the games chassis (#440)

The largest block: **1,479 lines, 52 names**. Not contiguous — interleaved with #388's lookup limits and #406's topic builder — so the boundaries came from a dependency pass over every top-level name rather than from reading the file.

The quiz went in with the games, against the ticket's own table, because it grades through `_graded_answers()`, picks topics through `_render_picker()` and reads `QUIZ_LANGS` beside `_fill_the_gap_round()`. Splitting them would have put an import between halves of one mechanism.

Tidied on the way past: `GAME_ROUNDS`'s explanatory comment had been sitting **280 lines above the declaration it describes**, on the far side of the entire #406 block.

Step 7 — card routes and icons (#441)

1,298 lines into `cards.py`, 94 into `icons.py`, and 8 lines of SSE framing into `web.py`.

This step reversed the ticket's stated order. #418 says chat then cards. Mykola's card saver *is* a card save — `_save_card_from_chat` needs `_save_and_log()` and `duplicate_notice()`, `get_mykola()` needs `DEFAULT_TOPIC` — so `chat.py` has to be able to import `cards`, and a module cannot import one that does not exist yet. Measured before choosing: the chat block crossed into `cards` four times and `cards` crossed back never.

Step 8 — Mykola (#443)

882 lines: the agent import, the `_mykola_agent` singleton, the chat routes, the savers the agent calls back into, the per-user chat logs.

Measured **self-contained in both directions** before it moved — which is exactly what moving `cards.py` first bought.

Nine names in `app.py` and `chat.py` were still reaching `web.py` through copies bound at import, left from #435 before the convention existed. Routing them through the module is what took the suite from 67 failures to green — not any change of behaviour.

Follow-up — scripts/ (#444, ticket #442)

Six console one-offs out of the root: `apply_schema.py`, `seed_topics.py` + `seed_words.py`, `claim_topics.py`, `claim_flashcards.py`, `retopic.py`.

They can move where the app's modules cannot, and that difference is the whole argument of #442: these are **run**, never imported by the site, so nothing resolves a data path relative to them at request time. The app's modules hold **seven** `Path(__file__)` sites that would fail *silently* one level down.

All five were **executed** from the new location against the real local MySQL, not merely imported.

7. What went wrong

Four failures, three of them silent. None reached main.

7.1 A decorator left behind, rerouting a live endpoint

A decorated function's ast line number is the def, **not the decorator**. A block boundary drawn from it leaves `@app.route` behind — and the orphan then attaches to the *next* function.

It happened twice:

- **step 7:** `topics_json`'s decorator was left behind and produced a `SyntaxError` — loud, and lucky.
- **step 8:** two decorators were orphaned and both reattached silently. ``/mykola/chat`` was answering with ``test_db_connection``, and `inject_mykola`'s context processor had landed on a `before_request`. Neither was a syntax error. The suite could not see either.

Caught by counting every route, context processor and `before_request` across all six modules and comparing with main: **46 before, 46 after**. That check is now the documented procedure in `CLAUDE.md`.

7.2 The database tier went silent

Moving `apply_schema.py` into `scripts/` **disabled all 127 database tests**. Each `*_db.py` gates itself on `_prerequisites_met()`, which checks the environment *and* `SCRIPT.exists()` — so to `pytest`, "the file moved" and "the tier is switched off" are the same thing.

Nothing was red. The only signal was the run finishing in **60 seconds instead of four minutes**, noticed by eye.

	passed	skipped	time
broken	2,109	143	1:02
fixed	2,250	3	3:51

A missing script is a broken checkout, not an environment prerequisite, so it is now an assertion rather than a skip — derived from the `*_db.py` files, so it covers a path added tomorrow.

7.3 A module prefix shadowing a local variable

`topics_json` already had a local variable named `icons`. A blanket `topic_icon( to icons.topic_icon(` rewrite shadowed the module with the local and `/topics.json` began answering **500**. Three tests caught it within one run.

A scan of all six modules for locals shadowing an imported module name then found a second: `_cards_for_chat` had a local named `cards` — harmless that day, and a trap the moment that function moved to `chat.py` the following step. Renamed before it could bite.

7.4 A branch cut from a stale origin/main

In step 8 the test branch was created without fetching first, so it lacked step 7's already-merged test sweep. Time was spent re-deriving work that existed. No harm to the result; the lesson is to fetch before branching, and that a code PR merging does not mean its paired test PR did.

8. Tests this produced

Three files that did not exist before, all guarding failures the rest of the suite **cannot see**:

file	guards
------	--------

test_suite_stays_offline.py	no route opens a real database connection
test_web_is_the_shared_module.py	no shared module imports app; app.py reaches names through the module; feature modules imported for side effects only
test_scripts_run_from_their_directory.py	each script runs from the repo root; the database tier's paths resolve

Every assertion in them was **proved to fail first**, per the repo's standing convention. Two examples of why that matters:

- ``test_web_is_the_shared_module.py``: a bare `import app` in `web.py` leaves **all 2,082 other tests green** while the ticket's entire purpose is gone. So does a bare `is_admin()` in `app.py`. That file is the only thing that notices either.
- ``test_scripts_run_from_their_directory.py``: deleting one `import _bootstrap` leaves **2,095 tests passing**, because `conftest.py` puts both directories on `sys.path` — the script would only fail on the deployment, at the moment somebody is following the deploy instructions.

Three existing tests were found to be vacuous or misdirected

- ``test_the_vet_does_not_vouch_for_words_in_confirmed_words`` could not fail, for two independent reasons: it stubbed `remember_confirmed_word` with a bare assignment on a module nothing had read since #436, *and* it never seeded random, so no word was ever rejected and there was nothing to vouch for in the first place. Both fixed; verified by making the vet actually vouch.
- ``test_the_note_is_written_once`` checked a claim about the whole codebase by grepping one file. After the move that count was zero — one rename from passing for the wrong reason. It now counts across every module.
- ``test_the_app_loads_its_own_env_before_the_agents`` (#412) compared two string offsets inside `app.py`; with the agent import in `chat.py` there was nothing left to compare. It now asks a subprocess for `sys.modules` order. Worth stating plainly: **that invariant is now harder to break than it was** — two independent changes are needed, and each alone was verified to leave the test correctly passing.

9. Rules this produced, now written into CLAUDE.md

1. **Reach a module through its name** — `web.is_admin()`, never a copy bound at import. A bare call still answers correctly; it is simply the one site a stub can no longer reach.
 2. **``app.py`` imports feature modules for side effects only.** A `from rounds import ...` means something was left half-moved.
 3. **``web.py`` takes only what two or more feature modules need.**
 4. **Moving a block? Compare the routes** — before and after, across every module.
 5. **``scripts/`` holds what is run, not imported**, and each script opens with `import _bootstrap`, whose position is load-bearing.
-

10. Verification

Every step was verified the same way.

- **An AST pass** listing names each module uses but does not define. This is what found `oauth` (lowercase, missed by a regex) and `_current_email` in the first, reverted attempt — each of which had cost a full suite run to discover.
- **Both test tiers**, compared against main run on the same machine, so the numbers meant something.
- **A real-MySQL smoke walk** at every step — the part offline stubs cannot prove. By the last steps this was all 29 fillable GET routes, a ten-answer graded POST, `/topics.json`'s four keys, all nine games for both picker and round, both quiz endpoints, the worksheet, and Mykola: agent available, `docs/user-guide.md` found, widget rendered, chat pages answering.

Final: **2,109 offline, 2,250 with a real database, 3 skipped.**

11. What is left

- **#442** — the package and application factory. The directory is the cosmetic half; the factory is the half that buys something, since `web.py` builds the app at import and that is why `conftest.py` is shaped as it is. The seven `Path(__file__)` sites that would fail *silently* under it are written down on the ticket. **Not before the demo.**
 - **#338** — SM-2 and long-term recall. Both design decisions are settled: keyed on `(user_id, card_id)`, verified results only. #416 already put the one write site in place: the six rounds that grade against a card row all go through `rounds._graded_answers()`, so per-learner recall is one write there rather than one per round.
-

Report generated 19 September 2026.